

Teoría de la Computación, 2021-1

Nota de clase 2: Gramáticas Regulares

Manuel Soto Romero

16 de marzo de 2021
Colegio de Ciencia y Tecnología UACM

2.1. Introducción a las gramáticas y lenguajes formales

Como concluimos en la sección anterior, una vez que definimos un lenguaje formal, nuestra mayor preocupación será si una cadena pertenece o no a este. Dicho de otra forma, queremos ver si una cadena está bien escrita de acuerdo a dicho lenguaje. Un ejemplo de lenguaje formal que seguramente conociste en el curso de Matemáticas Discretas es el Lenguaje de la Lógica Proposicional, mejor conocido como LPROP. Por cierto, es común que a las cadenas también se les llamen *expresiones*.

El alfabeto de la Lógica proposicional es el siguiente:

- *Constantes lógicas:* $\perp, \top$ (Otra notación es la de usar 0 y 1 o también F y T)
- *Variables proposicionales:* $p, q, r, s, \dots, p_1, q_2, \dots$
- *Conectivos lógicos:* $\neg, \vee, \wedge, \rightarrow, \leftrightarrow$
- *Paréntesis:* $(,)$

Con este alfabeto, el Lenguaje de la Lógica Proposicional (LPROP) se define de la siguiente manera:

1. Las constantes lógicas son expresiones de LPROP.
2. Las variables proposicionales son expresiones de LPROP.
3. Si φ es una expresión de LPROP, entonces $(\neg\varphi)$ lo es también (*negación*).
4. Si φ y ψ son expresiones de LPROP, entonces las siguientes también lo son (*operadores lógicos*):
 - a) $(\varphi \vee \psi)$ *disyunción*
 - b) $(\varphi \wedge \psi)$ *conjunción*
 - c) $(\varphi \rightarrow \psi)$ *condicional*
 - d) $(\varphi \leftrightarrow \psi)$ *bicondicional*
5. Son todas.

A partir de este Lenguaje Formal, veamos si las siguientes cadenas (expresiones) forman parte de él.

1. $(\neg p)$
 - Por la regla 2, p está LPROP puesto que es una variable proposicional.
 - Por la regla 3 y dado que p está LPROP, entonces $(\neg p)$ está LPROP.
2. $(q \wedge r)$
 - Por la regla 2, tanto q como r están en LPROP
 - Por la regla 4b, y dado que q y r están en LPROP, entonces $(q \wedge r)$ está en LPROP.
3. $((\neg p) \rightarrow (q \wedge r))$
 - a) Como vimos anteriormente, p, q y r están en LPROP.
 - b) De la misma forma $(\neg p)$ está en LPROP.
 - c) Igualmente vimos que $(q \wedge r)$ está en LPROP.
 - d) Por la regla 4c y porque tanto $(\neg p)$ y $(q \wedge r)$ están en LPROP, entonces $((\neg p) \rightarrow (q \wedge r))$ está en LPROP.
4. $((p \wedge q) \wedge r) \rightarrow (\neg p)$
 - a) Como vimos anteriormente, p, q y r están en LPROP.
 - b) Por la regla 4b y porque p y q están en LPROP, entonces $(p \wedge q)$ está en LPROP.
 - c) Por la regla 4b y porque $(p \wedge q)$ y r están en LPROP, entonces $((p \wedge q) \wedge r)$ está en LPROP.
 - d) Como vimos anteriormente $(\neg p)$ está en LPROP.
 - e) Por la regla 4c y porque $((p \wedge q) \wedge r)$ y $(\neg p)$ están en LPROP, entonces $((p \wedge q) \wedge r) \rightarrow (\neg p)$ está en LPROP.

Ejercicio 2.1. Se deja como ejercicio al lector la verificación de que la fórmula $(\neg(p \wedge (q \vee r)))$ siguiente está en LPROP:

2.1.1. Gramáticas

Otra forma de mostrar cómo se construyen las expresiones es mediante lo que se conoce como *gramática formal*, que es un mecanismo sencillo de especificación de reglas de construcción, llamadas *producciones* o *reglas de reescritura*, con las cuales se pueden generar expresiones de un lenguaje.

Las producciones tienen la siguiente forma:

símbolo ::= cadena

El símbolo $::=$ se lee como “se reescribe como” y al aplicar una regla particular sustituimos el lado izquierdo de este símbolo por la cadena que se encuentra del lado derecho. Usando esta notación podemos describir el lenguaje de la lógica proposicional con la siguiente gramática:

$$\begin{aligned}
S &::= E \\
E &::= \text{var} \\
E &::= \text{const} \\
E &::= \triangleright E \\
E &::= E \star E \\
E &::= (E) \\
\text{var} &::= p \mid q \mid \dots \\
\text{const} &::= \perp \mid \top \\
\triangleright &::= \neg \\
\star &::= \vee \mid \wedge \mid \rightarrow \mid \leftrightarrow
\end{aligned}$$

A una colección de reglas de reescritura como la anterior, le llamamos *gramática* porque nos describe la *forma* o *reglas sintácticas* que deben tener las expresiones lógicas bien construidas. Para producir expresiones (cadenas) correctas de acuerdo a la gramática, empezamos con el símbolo de la izquierda del $::=$ de la primera regla, y utilizamos las reglas de reescritura, que nos dicen que en cualquier momento podemos sustituir parte de (o toda) la expresión, si en ella localizamos una subexpresión que corresponda al lado izquierdo de cualquiera de las reglas y la sustituimos por el lado derecho. Veamos algunos ejemplos.

- Proceso de generación de la expresión p
 - S
 - E
 - var
 - p
- Proceso de generación de la expresión $(\top \vee \perp)$
 - S
 - E
 - (E)
 - $(E \star E)$
 - $(\text{const} \star E)$
 - $(\text{const} \vee E)$
 - $(\text{const} \vee \text{const})$
 - $(\top \vee \text{const})$
 - $(\top \vee \perp)$
- Proceso de generación de la expresión $\neg(p \wedge (q \vee r))$
 - S
 - E
 - $\triangleright E$
 - $\neg E$
 - $\neg(E)$
 - $\neg(E \star E)$
 - $\neg(\text{var} \star E)$

- $\neg(\text{var} \wedge E)$
- $\neg(\text{var} \wedge (E))$
- $\neg(p \wedge (E))$
- $\neg(p \wedge (E \star E))$
- $\neg(p \wedge (\text{var} \star E))$
- $\neg(p \wedge (\text{var} \vee E))$
- $\neg(p \wedge (\text{var} \vee \text{var}))$
- $\neg(p \wedge (q \vee \text{var}))$
- $\neg(p \wedge (q \vee r))$

Una secuencia de aplicación de reglas de reescritura, como las de los ejemplos anteriores, se conocen como *derivación* de una expresión (cadena); esta expresión es la que figura en el último de los pasos. Estas derivaciones pueden presentarse así o gráficamente utilizando un *árbol*.

Ejercicio 2.2. Dada la siguiente gramática que define paréntesis bien balanceados:

$$\begin{aligned} E &::= () \\ E &::= (E) \\ E &::= EE \end{aligned}$$

Realiza la derivación de las siguientes expresiones:

- $(())$
- $(() ())$

2.1.2. Árboles de derivación

Para representar arboles de derivación, consideramos lo siguiente:

- Cada nodo tiene un símbolo asociado.
- Para que de un nodo salgan aristas se requiere que el símbolo que está en ese nodo aparezca del lado izquierdo de alguna producción.
- Las aristas apuntan a cada uno de los símbolos que aparecen del lado derecho de la producción utilizada.
- Es importante observar que un nodo tiene un único símbolo asociado.
- Si el símbolo en un nodo se reescribe en una sucesión de tres símbolos, deberán salir tres líneas de él, una por cada símbolo.
- Aquellos nodos de los que no salen líneas son llamados *hojas*.

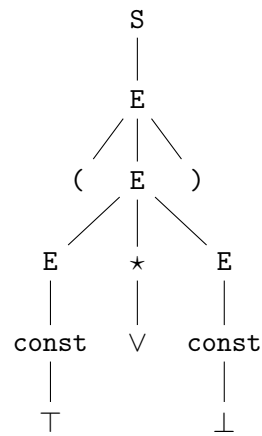
- Para determinar cuál es la expresión (cadena) que corresponde a un determinado árbol, vamos listando las hojas del árbol de izquierda a derecha. A la cadena que se obtiene de esta manera le vamos a llamar el *resultado del árbol*.

Veamos algunos ejemplos.

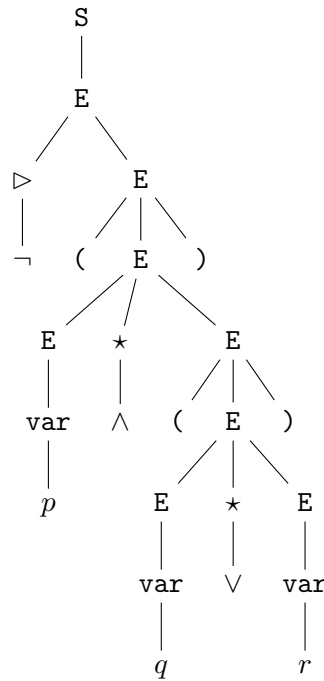
- p



- $(\top \vee \perp)$



- $\neg(p \wedge (q \vee r))$



Ejercicio 2.3. Realiza la derivación de las expresiones del Ejercicio 1.5.

2.1.3. La Jerarquía de Chomsky

La jerarquía de Chomsky es una clasificación jerárquica de distintos tipos de gramáticas que generan lenguajes formales. Esta jerarquía fue descrita por Noam Chomsky en 1956. La Jerarquía de Chomsky consta de cuatro niveles:

- **Gramáticas de tipo 0** (sin restricciones), que incluye a todas las gramáticas formales.
- **Gramáticas de tipo 1** (gramáticas sensibles al contexto) generan los lenguajes sensibles al contexto.
- **Gramáticas de tipo 2** (gramáticas libres del contexto) generan los lenguajes independientes del contexto.
- **Gramáticas de tipo 3** (gramáticas regulares) generan los lenguajes regulares.

A lo largo del curso estudiaremos cada uno de estos tipos de gramática, de momento basta con que sepas acerca de su existencia.

2.2. Lenguajes Regulares

Los lenguajes regulares se encuentran en la clase más pequeña que incluye a los lenguajes más simples. Por ejemplo, el conjunto de todos los números binarios. De hecho deben su nombre a la *regularidad* o repeticiones que ocurren entre sus cadenas. Por ejemplo:

$$L_1 = \{abc, cc, abab, abccc, ababc\}$$

La regularidad consiste en que sus palabras inician con repeticiones de ab seguidas de repeticiones de c . De hecho, los lenguajes finitos son también lenguajes regulares por definición:

$$L_2 = \{el, coche, verde\}$$

La combinación de lenguajes regulares (unión o concatenación) también producen un lenguaje regular. En general, diremos que un lenguaje L es regular si y sólo si se cumple al menos una de las siguientes condiciones:

- L es finito
- L es la unión o la concatenación de otros lenguajes regulares R_1 y R_2 , $L = R_1 \cup R_2$ o $L = R_1 R_2$ respectivamente.
- L es la cerradura de Kleene de algún lenguaje regular, $L = R^*$.

Por ejemplo, sea el lenguaje L de palabras formadas por a y b que empiezan con a , tales como $aab, ab, a, abaa$, etc. Podemos probar que este lenguaje es regular como sigue:

El alfabeto es $\Sigma = \{a, b\}$. El lenguaje L puede ser visto como la concatenación de una a con cadenas cualesquiera de a y b ; ahora bien, éstas últimas son elementos de $\{a, b\}^*$, mientras que el lenguaje que sólo contiene la palabra a es $\{a\}$. Ambos lenguajes son regulares, es decir, $\{a\}$ es finito, por lo tanto regular, mientras que $\{a, b\}^*$ es la cerradura de $\{a, b\}$, que es regular por ser finito también. Entonces la concatenación es $\{a\} \{a, b\}^*$, es regular.

En esta nota estudiaremos a los lenguajes regulares mediante algunos mecanismos como son las gramáticas, los autómatas finitos y las expresiones regulares.

2.3. Gramáticas Regulares

Aunque en la nota pasada usamos la notación $S := E$ para representar las reglas de producción, ésta es más utilizada en el ámbito del diseño de lenguajes de programación. En Teoría de la Computación se prefiere la notación $S \rightarrow E$.

Decimos entonces que las gramáticas regulares son gramáticas cuyas reglas son de la forma $A \rightarrow aB$ o bien $A \rightarrow a$, donde A y B son símbolos no terminales y a es un símbolo terminal. Por ejemplo:

$$\begin{aligned} S &\rightarrow aA \\ S &\rightarrow bA \\ A &\rightarrow aB \\ A &\rightarrow bB \\ A &\rightarrow a \\ B &\rightarrow aA \\ B &\rightarrow bA \end{aligned}$$

En este caso, dado que en los símbolos no terminales se encuentran a la derecha, decimos que es una *gramática regular derecha*, y cuando colocamos el símbolo no terminal a la izquierda, diremos evidentemente que, es *regular izquierda*. Por ejemplo:

$$\begin{aligned} S &\rightarrow Aa \\ S &\rightarrow Ab \\ A &\rightarrow Ba \\ A &\rightarrow Bb \\ A &\rightarrow a \\ B &\rightarrow Aa \\ B &\rightarrow Ab \end{aligned}$$

Definición 2.1. (*Gramática Regular*) Una gramática regular es una cuadrupla $\langle V, \Sigma, R, S \rangle$ en donde:

- V es el alfabeto de variables
- Σ es el alfabeto de constantes
- R es el conjunto de reglas de producción que además es un subconjunto de $V \times (\Sigma V \cup \Sigma)$
- S es el símbolo inicial, un elemento de V

Derivación

- Una cadena uXu deriva en un paso una cadena uav , escrito como $uXv \Rightarrow uav$, si hay una regla de producción $X \rightarrow \alpha \in R$ en la gramática.
- Una cadena $w \in \Sigma^*$ (formada sólo por símbolos terminales) es derivable a partir de una gramática G si existe una secuencia de pasos de derivación $S \Rightarrow \alpha_1 \Rightarrow \dots \Rightarrow w$.
- A una secuencia de pasos de derivación le llamamos simplemente derivación.
- Dicho de otra forma, una palabra $w \in \Sigma^*$ es derivable a partir de G si podemos llegar a ella a partir de S y escribimos

$$S \Rightarrow^* w$$

Definición 2.2. (*Lenguaje de una Gramática*) El Lenguaje generado por una gramática G , $L(G)$, es igual al conjunto de las cadenas derivables a partir de su símbolo inicial. Es decir:

$$L(G) = \{w \in \Sigma^* : S \Rightarrow^* w\}$$

Por ejemplo, el lenguaje reconocido por nuestra primera gramática es el de las palabras en $\{a, b\}$ de longitud par terminadas en a . Veamos otro ejemplo.

Ejemplo 2.1. Diseñar una gramática que genere el lenguaje de las palabras en $\{a, b\}$ que contiene la subcadena bb , tales como abb , $ababba$, etc.

Solución:

Podemos usar los símbolos no terminales para *recordar* situaciones. Es decir:

- Un símbolo A que recuerda que aún no se ha producido ninguna b .
- Un símbolo B que recuerda que se produjo una b .
- Un símbolo C que recuerda que ya se produjeron las dos b .

$$\begin{aligned} A &\rightarrow aA \\ A &\rightarrow bB \\ B &\rightarrow aA \\ B &\rightarrow bC \\ B &\rightarrow b \\ C &\rightarrow aC \\ C &\rightarrow bC \\ C &\rightarrow a \\ C &\rightarrow b \end{aligned}$$

Ejercicio 2.4. Diseña una gramática que dado el siguiente alfabeto $\{a, b, c, 1, 2, 3\}$ permita construir identificadores válidos. Decimos que un identificador es válido si éste es una combinación de letras y números pero con la restricción de que nunca debe empezar por un número. Por ejemplo abc , $a12$, $a1b$.

Referencias

- [1] Favio E. Miranda, Elisa Viso, *Matemáticas Discretas*, Las Prensas de Ciencias, Facultad de Ciencias UNAM, 2010.